Technical Report

Gradient Estimation for Mixture Distributions

Abstract: Finite mixture distributions provide flexible posterior approximations whose optimization requires gradients through both component parameters and mixture weights. We review the main approaches to this problem and describe four unbiased estimators: score-function, stratified, implicit reparameterization, and post-stratified. For each method, we explain its construction, requirements, and computational cost, including the Sticking the Landing modification for variational inference. These estimators will form ReMix, a PyTorch library with a shared interface for optimization and comparison.

1 Introduction

Variational inference approximates a posterior by optimizing a tractable density. Finite mixtures make the approximation more expressive by representing several modes with separate components. Variational boosting adds these components incrementally [1], while automatic differentiation variational inference with mixtures jointly optimizes their parameters and weights [2].

Sampling a mixture involves drawing a categorical index and then a continuous value from the chosen component. Training requires expectation gradients that account for parameter changes in both stages. Score-function estimators use derivatives of the log density [3], while pathwise estimators differentiate the sampled value, as in variational autoencoders [4, 5]. Each approach has its own variance and computational requirements [6].

Several methods adapt these ideas to mixtures. Stratification samples every component and combines the resulting gradients using the mixture weights [2, 7]. Implicit reparameterization obtains sample derivatives from distribution functions [8, 9]. Transport-based methods describe how probability mass moves as parameters change [10, 11]. This perspective also leads to a post-stratified estimator, which averages component derivatives at a single mixture sample. Burroni and Sheldon [12] bring these approaches into a common formulation and analyze their computational and variance properties.

We will implement four algorithms as the ReMix library: a score-function baseline, stratification, implicit reparameterization, and post-stratification. The library will provide a shared PyTorch interface, starting with mixtures of diagonal Gaussian components, and support the Sticking the Landing (STL) modification for evidence lower bound gradients [7]. This report specifies the estimators and their implementation requirements as a basis for comparing gradient variance, optimization progress, and runtime.

2 Related Work

Methods for discrete random variables provide further ways to handle a mixture's categorical selector. Concrete and Gumbel–Softmax use a smooth, temperature-controlled approximation and differentiate the resulting relaxed objective [13, 14]. REBAR uses a continuous relaxation as a control variate with a correction that preserves unbiasedness for the discrete objective [15]. RELAX learns a differentiable control variate jointly with the model [16]. ARSM constructs coupled categorical samples through augmentation and category swaps, sharing randomness to reduce gradient variance [17]. These approaches offer alternatives for optimizing mixture selection probabilities.

Other work extends pathwise differentiation to broader continuous families. Generalized reparameterization combines a sample derivative with a score correction for parameter-dependent transformed noise [18]. Naesseth et al. [19] derive gradients through acceptance–rejection samplers, enabling applications to distributions such as Gamma and Dirichlet. These methods provide useful directions for extending the component families supported by ReMix. For the initial implementation, an existing PyTorch package for implicit mixture gradients provides a software reference [20]; the discussion of mixture sampling derivatives in PyTorch issue #39178 gives additional implementation context [21].

3 Problem Setup

Let $\theta \in \mathbb{R}^m$ contain the parameters of a mixture density on $\mathbb{R}^d$. Our objective and mixture are

$$\mathcal{J}(\theta) = \mathbb{E}_{q_\theta}[f_\theta(Z)], \qquad q_\theta(z) = \sum_{k=1}^{K} \pi_k(\theta) q_{k,\theta}(z), \qquad \pi_k > 0, \quad \sum_k \pi_k = 1. \tag{1}$$

We assume differentiable densities that are positive on the interior of a common, parameter-independent support, with integrable derivatives allowing differentiation under the integral. Pathwise identities additionally require vanishing boundary flux; variance comparisons require finite second moments.

The responsibility $r_k(z) = \pi_k q_{k,\theta}(z)/q_\theta(z)$ is the conditional probability that component k generated z. We write $a_k = \nabla_\theta \log \pi_k$ for the weight score and $s_\theta(z) = \partial_\theta \log q_\theta(z)$ for the mixture score. For the objective, $h_\theta(z) = \nabla_z f_\theta(z)$ describes its sensitivity to the sample, while $d_\theta(z) = \partial_\theta f_\theta(z)$ captures direct parameter dependence at fixed z.

Component samples take the form $Z_k = T_{k,\theta}(\varepsilon_k)$, where $\varepsilon_k \sim \rho$ has a parameter-independent distribution. Stratification uses derivatives of these maps. Post-stratification also uses their inverses; for this method, we assume differentiable component bijections onto the common support and write $V_k(z)$ for $D_\theta T_{k,\theta}$ evaluated at $\varepsilon_k = T_{k,\theta}^{-1}(z)$.

For a fixed model $p(x, z)$, variational inference maximizes the evidence lower bound (ELBO), with integrand

$$f_\theta(z) = \log p(x, z) - \log q_\theta(z), \qquad d_\theta(z) = -s_\theta(z). \tag{2}$$

The score has zero expectation, so omitting d_θ preserves the expected ELBO gradient. This is the STL modification [7]. In automatic differentiation, it freezes the parameters of q inside the log-density term while preserving its derivative through z. We denote an operation that preserves a value and stops its derivative by sg. For general objectives, the estimators retain d_θ.

4 Gradient Estimators

The following estimators target $\nabla_\theta \mathcal{J}$ under the assumptions above, with optional STL for the ELBO.

Algorithms 1–4 return an objective estimate and a scalar surrogate $\mathcal{S}$; differentiating $\mathcal{S}$ gives the estimated ascent gradient. A loss-minimizing optimizer uses $-\mathcal{S}$. Write $\varphi_\theta(z) = f_\theta(z)$ for ordinary differentiation. With STL, use $\varphi_\theta(z) = \log p(x, z) - \log q_{\mathrm{sg}(\theta)}(z)$, retaining the derivative through z. For diagonal Gaussian components, $\rho = \mathcal{N}(0, I)$ and $T_{k,\theta}(\varepsilon) = \mu_k + \sigma_k \odot \varepsilon$. The single-sample algorithms first select a component with probabilities π, then draw its continuous value. Independent repetitions can be averaged to increase the sample budget.

4.1 Score-function estimator

The score-function method differentiates the mixture density at a fixed sample. For $Z \sim q_\theta$ and a baseline b independent of Z, it gives [3, 6]

$$\widehat{g}_{\mathrm{SF}}(Z) = (f_\theta(Z) - b)s_\theta(Z) + d_\theta(Z). \tag{3}$$

The identity follows by differentiating under the integral. The baseline can be chosen to reduce variance because its contribution has zero expectation. The method requires a differentiable mixture log density.

Algorithm 1 draws one component and its base noise, then detaches the resulting value. Differentiating the log density supplies the score term, while differentiating $\varphi_\theta(z_0)$ supplies the direct objective term. The baseline is detached in the score coefficient. With softmax logits α and component parameters independent of α, the weight gradient is $(f_\theta(Z) - b)(r_j(Z) - \pi_j)$ for logit α_j with STL.

Algorithm 1: Score-function sampling and surrogate

Input: Mixture q_θ, component maps $T_{k,\theta}$, objective φ_θ, baseline b

Draw $c \sim \mathrm{Cat}(\pi)$ and $\varepsilon \sim \rho$;
$z_0 \leftarrow \mathrm{sg}(T_{c,\theta}(\varepsilon))$;
$v \leftarrow \varphi_\theta(z_0)$; $\quad \ell \leftarrow \log q_\theta(z_0)$;
$\mathcal{S} \leftarrow \mathrm{sg}(v - b)\,\ell + v$;
return $(\mathrm{sg}(v), \mathcal{S})$;

4.2 Stratified estimator

Stratification evaluates the objective within every component and combines the results using the mixture weights. Draw $N_k \geq 1$ independent base samples for component k, with each N_k fixed, and set $Z_{k,n} = T_{k,\theta}(\varepsilon_{k,n})$. Differentiating the weighted component expectations gives [2, 7]

$$\widehat{g}_{\mathrm{ST}} = \sum_k \frac{\pi_k}{N_k} \sum_{n=1}^{N_k} \left[f_\theta(Z_{k,n})a_k + D_\theta T_{k,\theta}(\varepsilon_{k,n})^\top h_\theta(Z_{k,n}) + d_\theta(Z_{k,n}) \right]. \tag{4}$$

In practice, automatic differentiation of the weighted sample average constructs all three terms. Algorithm 2 draws independent noise within every component and retains the derivative through each transformation and the outer mixture weights. Every sampled point is evaluated using the full mixture density in the ELBO.

Algorithm 2: Stratified sampling and surrogate

Input: Weights π, component maps $T_{k,\theta}$, objective φ_θ, fixed counts $N_k \geq 1$

$\mathcal{S} \leftarrow 0$;
for $k = 1, \ldots, K$ **do**
  for $n = 1, \ldots, N_k$ **do**
    Draw independent $\varepsilon_{k,n} \sim \rho$;
    $z_{k,n} \leftarrow T_{k,\theta}(\varepsilon_{k,n})$;
    $\mathcal{S} \leftarrow \mathcal{S} + (\pi_k / N_k)\varphi_\theta(z_{k,n})$;
  end
end
return $(\mathrm{sg}(\mathcal{S}), \mathcal{S})$;

Sampling every component removes component-selection randomness and allows the sample budget to be allocated across components. One sample per component requires K objective evaluations. For a chain of T conditional mixtures, full enumeration requires $\prod_{t=1}^{T} K_t$ evaluations, where K_t is the component count at layer t. This growth motivates single-sample methods.

4.3 Implicit reparameterization

Implicit reparameterization differentiates an exact mixture sample through a standardization S_θ, a differentiable bijection mapping $Z \sim q_\theta$ to a fixed reference distribution. Holding the standardized value constant gives [8, 9]

$$A_\theta(z)V_{\mathrm{IR}}(z) = -D_\theta S_\theta(z), \qquad A_\theta(z) = D_z S_\theta(z), \qquad \widehat{g}_{\mathrm{IR}}(Z) = V_{\mathrm{IR}}(Z)^\top h_\theta(Z) + d_\theta(Z). \tag{5}$$

The matrix A_θ must be nonsingular, and the inverse transformation must satisfy the differentiation assumptions. In one dimension, S_θ is the mixture cumulative distribution function (CDF), so the sample derivative is $-\partial_\theta F_\theta(z)/q_\theta(z)$.

For a multivariate mixture, a suitable standardization is the Rosenblatt transform: coordinate i is the conditional CDF $S_{\theta,i}(z) = F_\theta(z_i \mid z_{<i})$. Its component weights are proportional to $\pi_k q_{k,\theta}(z_{<i})$, accounting for the information in preceding coordinates. For factorized components, the prefix density is a product of univariate densities, and the component conditional CDF is its univariate CDF. The resulting Jacobian is lower triangular with positive conditional densities on the diagonal, enabling a triangular solve.

Algorithm 3 first draws an exact sample through the categorical index and component map. It then evaluates the standardization at that detached point and attaches its implicit derivative through a linear solve. The corrected value $\widetilde{z}$ equals z_0 and carries the first-order implicit gradient. The method requires one objective evaluation, CDF derivatives, and a Jacobian solve. A generic reverse-mode construction costs $O(dC_S + d^2)$, where C_S is the cost of evaluating S_θ.

Algorithm 3: Implicit sampling and surrogate

Input: Weights π, component maps $T_{k,\theta}$, standardization S_θ, objective φ_θ

Draw $c \sim \mathrm{Cat}(\pi)$ and $\varepsilon \sim \rho$;
$z_0 \leftarrow \mathrm{sg}(T_{c,\theta}(\varepsilon))$;
$s \leftarrow S_\theta(z_0)$; $\quad A \leftarrow \mathrm{sg}(D_z S_\theta(z)|_{z=z_0})$;
Solve the triangular system $Au = -s$, retaining the derivative through s;
$\widetilde{z} \leftarrow z_0 + u - \mathrm{sg}(u)$;
$v \leftarrow \varphi_\theta(\widetilde{z})$; $\quad \mathcal{S} \leftarrow v$;
return $(\mathrm{sg}(v), \mathcal{S})$;

4.4 Post-stratified estimator

Post-stratification draws one exact mixture sample and evaluates how every component could have generated that value. It recovers the base noise through each inverse map and averages the component derivatives using responsibilities:

$$\overline{V}(z) = \sum_k r_k(z)V_k(z), \qquad \widehat{g}_{\mathrm{PS}}(Z) = \overline{V}(Z)^\top h_\theta(Z) + f_\theta(Z)\sum_k r_k(Z)a_k + d_\theta(Z). \tag{6}$$

The identity $\mathbb{E}_{q_\theta}[r_k(Z)H(Z)] = \pi_k \mathbb{E}_{q_{k,\theta}}[H(Z)]$ connects this expression to the stratified gradient and establishes unbiasedness [11, 12]. Component parameters receive pathwise derivatives, and trainable weights receive the score contribution. With independent softmax logits, the latter is $f_\theta(Z)(r_j(Z) - \pi_j)$.

Algorithm 4 draws one component to obtain z_0, then recovers a base-noise value for every component at that same point. For diagonal Gaussians, the inverse map is $T^{-1}_{k,\theta}(z) = (z - \mu_k) \oslash \sigma_k$. The recovered noises e_k and responsibilities $\bar{r}_k$ are detached before reconstructing the sample. The resulting surrogate is

$$\widetilde{z} = \sum_k \bar{r}_k T_{k,\theta}(e_k), \qquad \mathcal{S}_{\mathrm{PS}} = f_\theta(\widetilde{z}) + \mathrm{sg}(f_\theta(z_0))\sum_k \bar{r}_k \log \pi_k. \tag{7}$$

Each reconstructed component sample equals z_0, so $\widetilde{z}$ preserves the original value while carrying the averaged derivative. The objective value is reused in the detached weight correction. The choice of φ_θ applies ordinary differentiation or STL to the first term.

Algorithm 4: Post-stratified sampling and surrogate

Input: Mixture q_θ, component maps $T_{k,\theta}$ and their inverses, objective φ_θ

Draw $c \sim \mathrm{Cat}(\pi)$ and $\varepsilon \sim \rho$;
$z_0 \leftarrow \mathrm{sg}(T_{c,\theta}(\varepsilon))$;
for $k = 1, \dots, K$ **do**
  $e_k \leftarrow \mathrm{sg}(T_{k,\theta}^{-1}(z_0))$; $\bar{r}_k \leftarrow \mathrm{sg}(\pi_k q_{k,\theta}(z_0)/q_\theta(z_0))$;
end
$\widetilde{z} \leftarrow \sum_k \bar{r}_k T_{k,\theta}(e_k)$;
$v \leftarrow \varphi_\theta(\widetilde{z})$;
$\mathcal{S} \leftarrow v + \mathrm{sg}(v) \sum_k \bar{r}_k \log \pi_k$;
return $(\mathrm{sg}(v), \mathcal{S})$;

This conditional averaging is a Rao–Blackwellization: it has a smaller or equal covariance than using only the sampled component's pathwise and weight-score terms, including with STL [12]. It requires one objective evaluation, with all component densities and inverse maps evaluated at the same sample.

5 Planned Library

ReMix will expose the four estimators through an interface accepting a mixture and an objective. Diagonal Gaussian components will have trainable logits, means, and positive scales. Each estimator will return an objective estimate for logging and a scalar surrogate for automatic differentiation, with optional STL for the ELBO.

Table 1 summarizes the requirements. Each mixture density evaluation includes all K components; stratification repeats this work at every sample point. Shared operations will use log-sum-exp for mixture log densities, normalized component log densities for responsibilities, and triangular solves for implicit derivatives.

Tab. 1: Estimator requirements. Stratification uses one sample per component.

Estimator	Objective evaluations	Required operations
Score-function	1	Mixture log-density derivatives
Stratified	K	Component sampling derivatives
Implicit	1	Conditional CDFs and a Jacobian solve
Post-stratified	1	Inverse component maps and responsibilities

We will compare gradient variance, ELBO progress, runtime, and memory use across component counts, overlap levels, and event dimensions. Progress per model evaluation and per unit of time will measure practical optimization efficiency.

References

[1] Andrew C. Miller, Nicholas J. Foti, and Ryan P. Adams. Variational boosting: Iteratively refining posterior approximations. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2420–2429, 2017. URL https://proceedings.mlr.press/v70/miller17a.html.

[2] Warren Morningstar, Sharad Vikram, Cusuh Ham, Andrew Gallagher, and Joshua Dillon. Automatic differentiation variational inference with mixtures. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pages 3250–3258, 2021. URL https://proceedings.mlr.press/v130/morningstar21b.html.

[3] Rajesh Ranganath, Sean Gerrish, and David M. Blei. Black box variational inference. In *Proceedings of the Seventeenth International Conference on Artificial Intelligence and Statistics*, volume 33 of *Proceedings of Machine Learning Research*, pages 814–822, 2014. URL https://proceedings.mlr.press/v33/ranganath14.html.

[4] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014. URL https://arxiv.org/abs/1312.6114.

[5] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32(2) of *Proceedings of Machine Learning Research*, pages 1278–1286, 2014. URL https://proceedings.mlr.press/v32/rezende14.html.

[6] Shakir Mohamed, Mihaela Rosca, Michael Figurnov, and Andriy Mnih. Monte Carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132):1–62, 2020. URL https://www.jmlr.org/papers/v21/19-346.html.

[7] Geoffrey Roeder, Yuhuai Wu, and David K. Duvenaud. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1703.09194.

[8] Alex Graves. Stochastic backpropagation through mixture density distributions, 2016. URL https://arxiv.org/abs/1607.05690.

[9] Michael Figurnov, Shakir Mohamed, and Andriy Mnih. Implicit reparameterization gradients. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://arxiv.org/abs/1805.08498.

[10] Martin Jankowiak and Fritz Obermeyer. Pathwise derivatives beyond the reparameterization trick. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2235–2244, 2018. URL https://proceedings.mlr.press/v80/jankowiak18a.html.

[11] Martin Jankowiak and Theofanis Karaletsos. Pathwise derivatives for multivariate distributions. In *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 333–342, 2019. URL https://proceedings.mlr.press/v89/jankowiak19a.html.

[12] Javier Burroni and Daniel Sheldon. Gradient estimation for mixture variational inference. *Journal of Machine Learning Research*, 27(191):1–34, 2026. URL https://www.jmlr.org/papers/v27/25-2560.html.

[13] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1611.00712.

[14] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel–Softmax. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1611.01144.

[15] George Tucker, Andriy Mnih, Chris J. Maddison, Dieterich Lawson, and Jascha Sohl-Dickstein. REBAR: Low-variance, unbiased gradient estimates for discrete latent variable models. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1703.07370.

[16] Will Grathwohl, Dami Choi, Yuhuai Wu, Geoffrey Roeder, and David Duvenaud. Backpropagation through the void: Optimizing control variates for black-box gradient estimation. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1711.00123.

[17] Mingzhang Yin, Yuguang Yue, and Mingyuan Zhou. ARSM: Augment-REINFORCE-swap-merge estimator for gradient backpropagation through categorical variables. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 7095–7104, 2019. URL https://proceedings.mlr.press/v97/yin19c.html.

[18] Francisco J. R. Ruiz, Michalis K. Titsias, and David M. Blei. The generalized reparameterization gradient. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://arxiv.org/abs/1610.02287.

[19] Christian A. Naesseth, Francisco J. R. Ruiz, Scott W. Linderman, and David M. Blei. Reparameterization gradients through acceptance-rejection sampling algorithms. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 489–498, 2017. URL https://proceedings.mlr.press/v54/naesseth17a.html.

[20] vsimkus. torch-reparametrised-mixture-distribution. GitHub repository, n.d. URL https://github.com/vsimkus/torch-reparametrised-mixture-distribution. Accessed 21 October 2026.

[21] PyTorch contributors. Add support for `rsample` to MixtureSameFamily distribution. GitHub issue #39178, 2020. URL https://github.com/pytorch/pytorch/issues/39178. Accessed 21 October 2026.